Lecture No.03
Arrays and List

R.A: Assad Ullah Khan

in assadullahkhan

AssadUllahKhan

Arrays

- Imagine that we have 100 scores. We need to read them, process them and print them. We must also keep these 100 scores in memory for the duration of the program. We can define a hundred variables, each with a different name, as shown in Figure

Score 1

Score 2

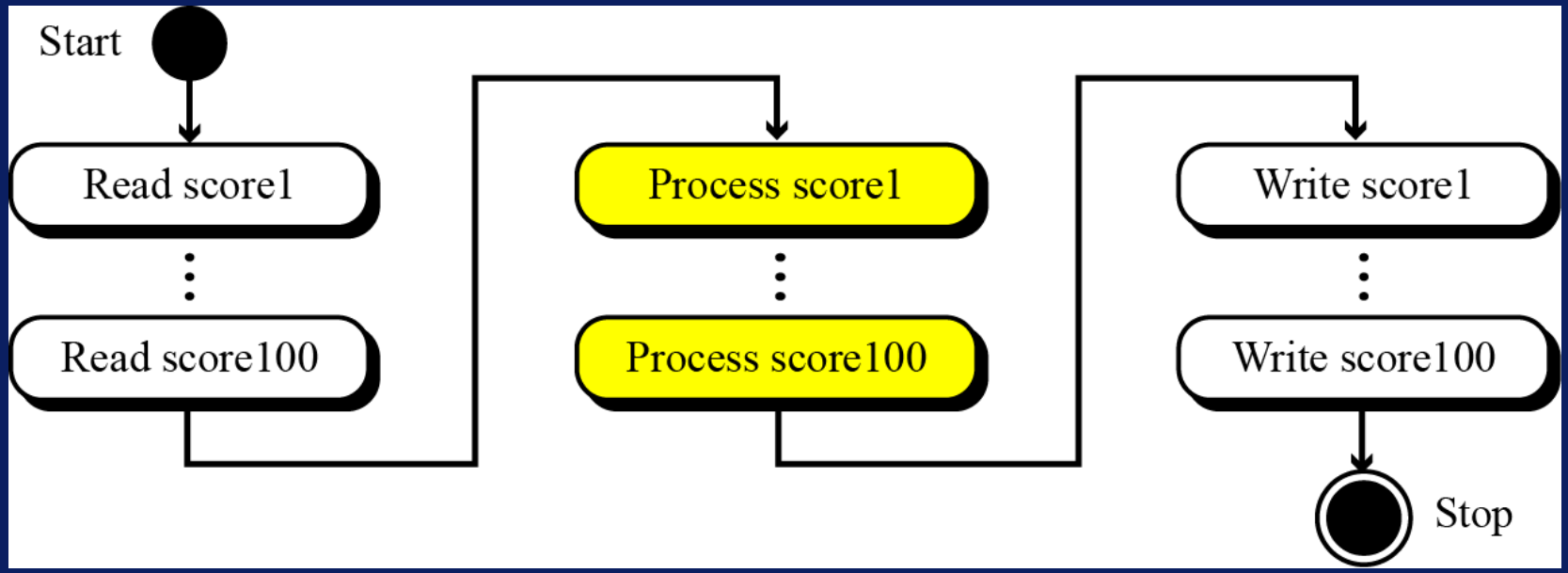
.

.

Score 100

Arrays

- But having 100 different names creates other problems. We need 100 references to read them, 100 references to process them and 100 references to write them. Following Figure shows a diagram that illustrates this problem



Arrays

- Array declaration: int x[6];
- An array is collection of cells of the same type.
- The collection has the name ‘x’.
- The cells are numbered with consecutive integers.
- In C, C++ and Java, index starts from 0
- To access a cell, use the array name and an index:

  x[0], x[1], x[2], x[3], x[4], x[5]

Arrays

- We can use loops to read and write the elements in an array & also use loops to process elements
- Now it does not matter if there are 100, 1000 or 10,000 elements to be processed—loops make it easy to handle them all
- We can use an integer variable to control the loop and remain in the loop as long as the value of this variable is less than the total number of elements in the array

Arrays

- Elementary data structure that exists as built-in in most programming languages.

```
void main(  )
{
    int x [ 100 ];
    int i;
    for( i = 0; i < 100; i++)
            x[ i ] = 2 * i;
}
```

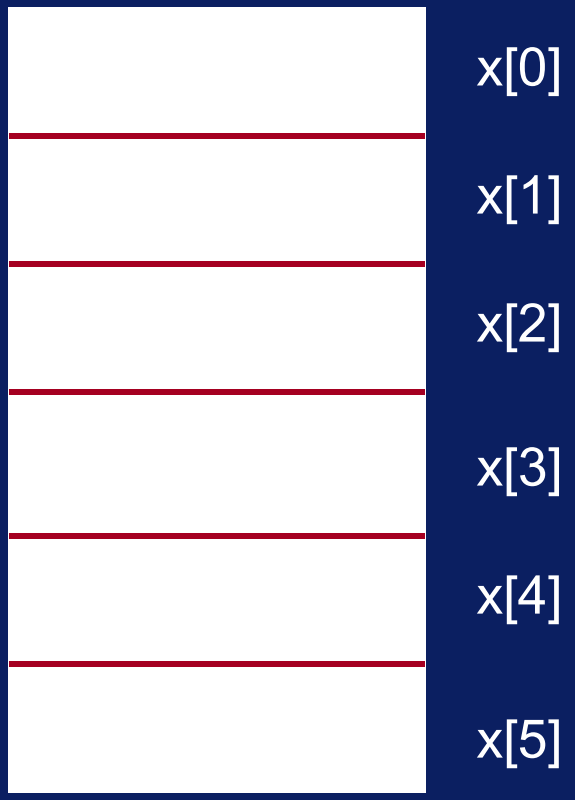
Arrays

```
int main()
{
    int arr[5];
    arr[0] = 1;
    arr[1] = 2;
    arr[2] = 3;
    arr[3] = 4;
    arr[4] = 5;
    printf("%d %d %d %d %d\n", arr[0], arr[1], arr[2],
      arr[3], arr[4]);
    return 0;
}
```

Array Layout

Array cells are contiguous in computer memory

The memory can be thought of as an array



What is Array Name?

- ‘x’ is an array name but there is no variable x. ‘x’ is not an *lvalue*.
- For example, if we have the code

```
int a, b;
```

then we can write

```
b = 2;
a = b;
a = 5;
```

But we cannot write

```
2 = a;
```

Array Name

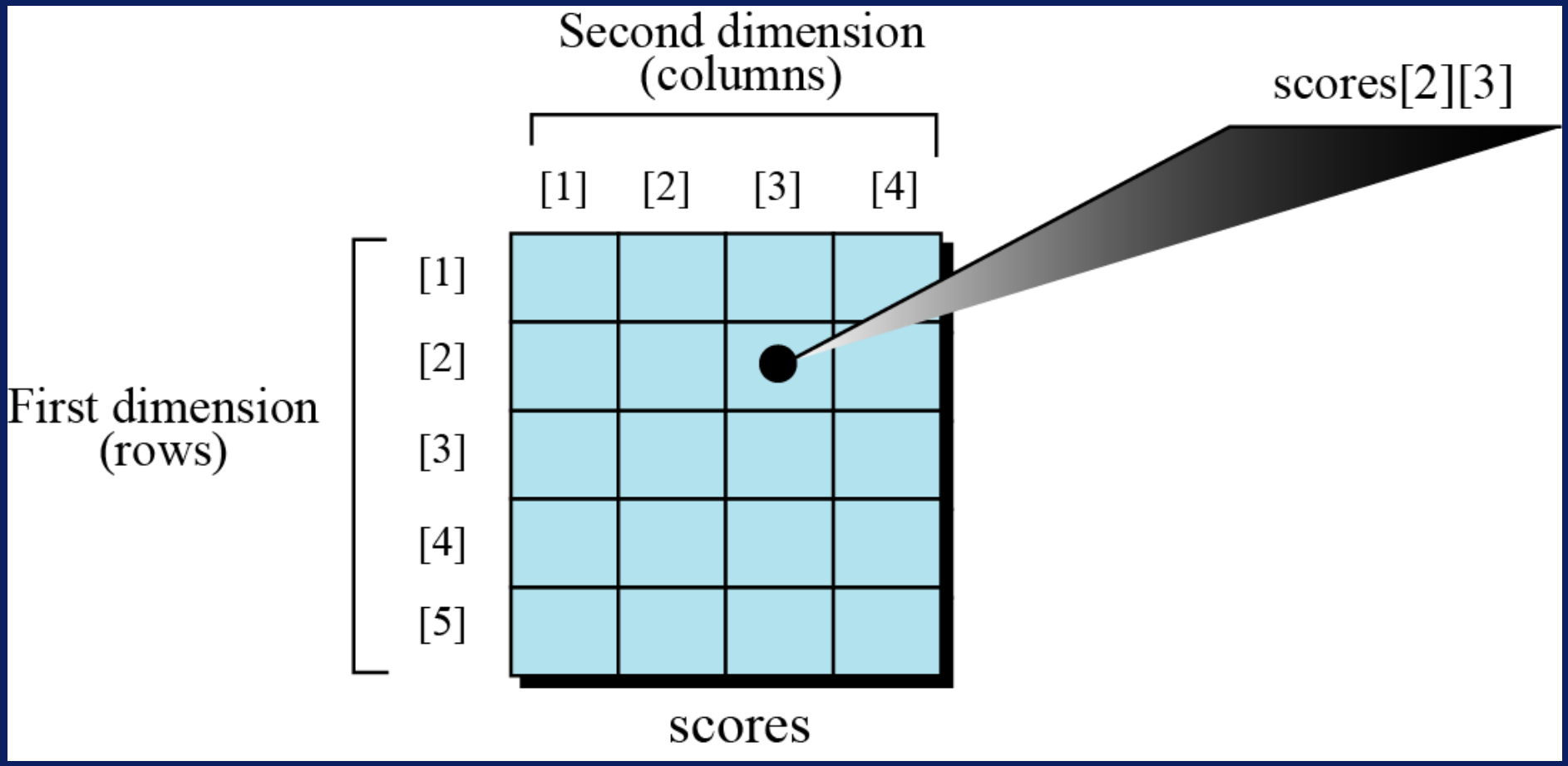
- ‘x’ is not an lvalue

```
int x[6];
int n;

x[0] = 5;
x[1] = 2;

x = 3;          // not allowed
x = a + b;      // not allowed
x = &n;         // not allowed
```

Multi-Dimensional Arrays

- The arrays discussed so far are known as one-dimensional arrays because the data is organized linearly in only one direction
- Many applications require that data be stored in more than one dimension
- In the next slide the Figure shows a table, which is commonly called a two-dimensional array

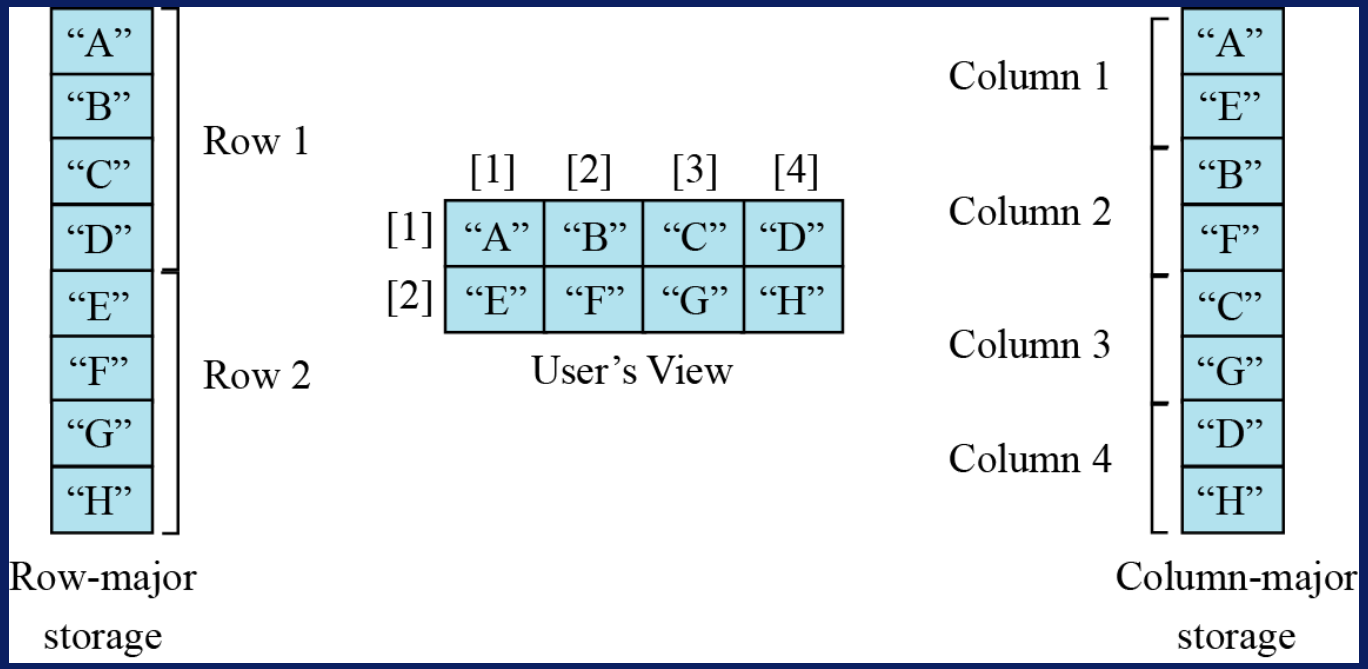
Multi-Dimensional Arrays



A two-dimensional array

```
int main()
{
    // 2 rows and 3 columns ka 2D array
    int arr[2][3];

    // Manually values assign kar rahe hain
    arr[0][0] = 1;
    arr[0][1] = 2;
    arr[0][2] = 3;
    arr[1][0] = 4;
    arr[1][1] = 5;
    arr[1][2] = 6;

    // Print karte hain array elements
    printf("%d %d %d\n", arr[0][0], arr[0][1], arr[0][2]);
    printf("%d %d %d\n", arr[1][0], arr[1][1], arr[1][2]);

    return 0;
}
```

- Let "Arr" be a two dimensional m x n array having m=2 and n=4, then m x n = 8 sequential locations Following Figure shows a two-dimensional array and how it is stored in memory using row-major or column-major storage.
  Row-major storage is more common



A two-dimensional array

Storage Allocation

The operating system does not keep the address of each element of the Array instead it only keeps the address of the 1st element and this address is known as Base Address. The address of any element may be found by using the formula

Low Addresses

Base Address

Index of Arr

Address of Arr [i]

i is the position of the element in Arr

Memory Address	Memory	Array Elements
1051		Arr [0]
1052		
1053		Arr [1]
1054		
1055		Arr [2]
1056		
1057		Arr [3]
1058		
1059		Arr [4]
1060		
1061		Arr [5]
1062		

address of Arr[i] = Base Address + W(i-LB)

* W is the word Length

* LB is Lower Bound

High Addresses

Storage Allocation

We have an array of int data type then W= 2 Bits, the address of the first element is 1051 and the 0 i.e LB=0

Calculate the address of 4th element in Array

Low Addresses

Base Address

Index of Arr

Address of Arr [i]

Memory Address	Memory	Array Elements
1051		Arr [0]
1052		
1053		Arr [1]
1054		
1055		Arr [2]
1056		
1057		Arr [3]
1058		
1059		Arr [4]
1060		
1061		Arr [5]
1062		

High Addresses

address of Arr[i] = Base Address + W(i-LB)

address of Arr[3] = 1051 + 2 (3 - 0)

address of Arr[3] = 1051 + 6

address of Arr[3] = 1057

The LIST Data Structure

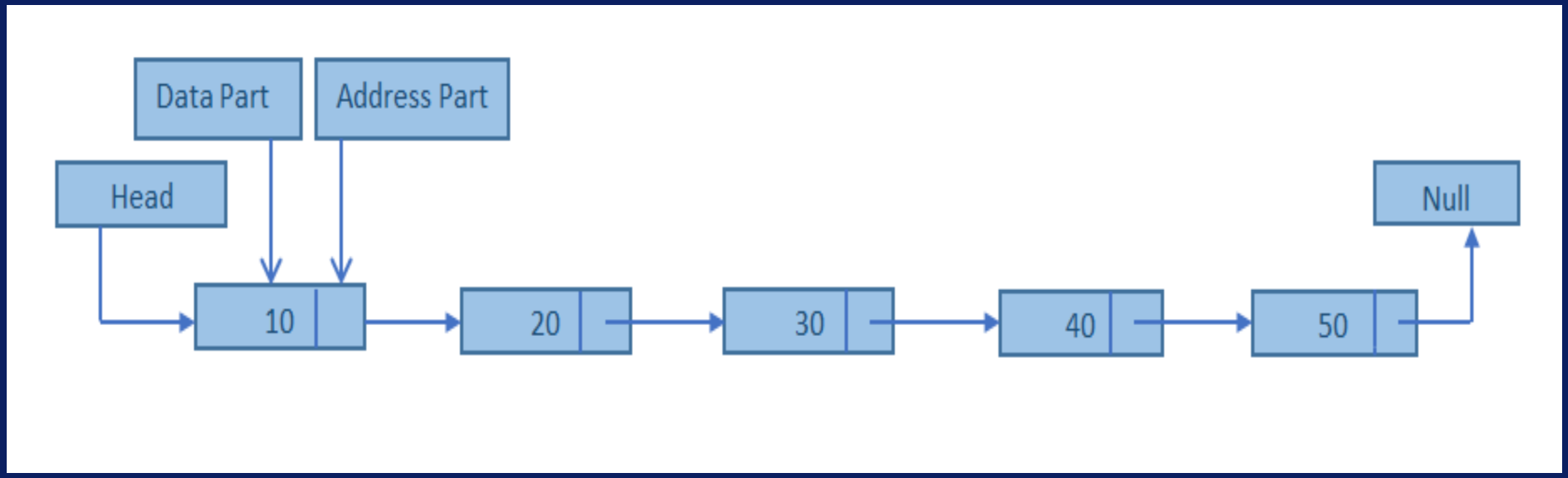
- The List is among the most generic of data structures.
- Real life:

  a. shopping list,
  b. groceries list,
  c. list of people to invite to dinner

Lists

- A list is collection of items that are all of the same type (grocery items, integers, names)
- linked list is a dynamic and linear data structure
- The items, or elements of the list, are stored in some particular order
- It is possible to insert new elements into various positions in the list and remove any element of the list
- Link list used for the dynamic memory allocation.
- Array and link list both are the linear data structure

Representation of Singly Link List or Chain Link List



Advantage of Link list

- Link list is an example of dynamic data structure
- They can grow and shrink during the execution of program
- Efficient memory utilization
- Memory is not pre allocated like static data structure
- The allocation of memory depends upon the user.
- Insertion and deletion easily performed

Points to Remember

- Linked lists are used when the quantity of data is not known prior to execution
- In linked lists, data is stored in the form of nodes and at runtime, memory is allocated for creating nodes
- Due to overhead in memory allocation and deallocation, the speed of the program is lower.
- The data is accessed using the starting pointer of the list

List Operations

Useful operations

- createList(): create a new list (presumably empty)
- copy(): set one list to be a copy of another
- clear(); clear a list (remove all elments)
- insert(X, ?): Insert element X at a particular position in the list
- remove(?): Remove element at some position in the list
- get(?): Get element at a given position
- update(X, ?): replace the element at a given position with X
- find(X): determine if the element X is in the list
- length(): return the length of the list.

List Operations

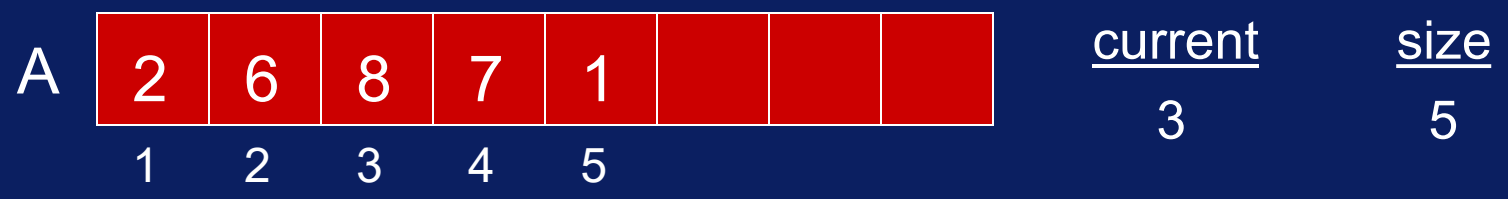
- If we use the “current” marker, the following four methods would be useful:
  - **start()**: moves to “current” pointer to the very first element.
  - **tail()**: moves to “current” pointer to the very last element.
  - **next**(): move the current position forward one element
  - **back**(): move the current position backward one element

Implementing Lists

- We have designed the interface for the List; we now must consider how to implement that interface.

Implementing Lists

- We have designed the interface for the List; we now must consider how to implement that interface.
- Implementing Lists using an array: for example, the list of integers (2, 6, 8, 7, 1) could be represented as:



List Implementation

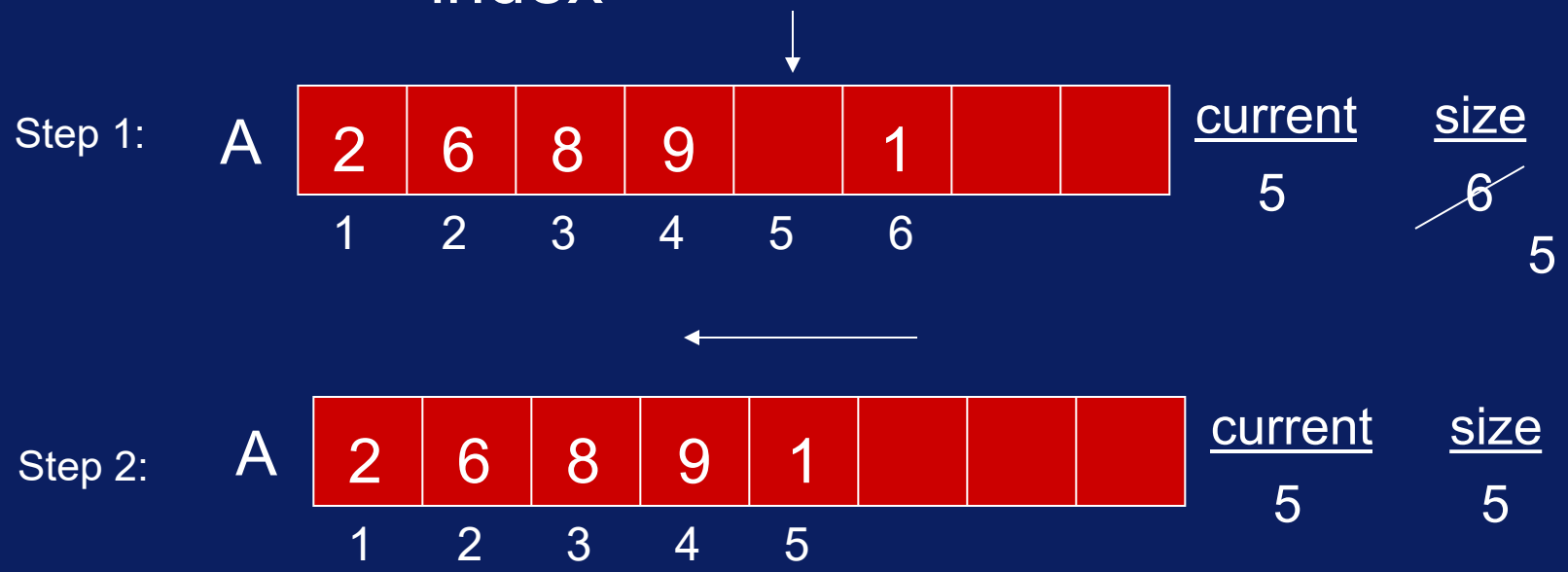
- add(9); current position is 3. The new list would thus be: (2, 6, 8, 9, 7, 1)
- We will need to *shift* everything to the right of 8 one place to the right to make place for the new element ‘9’.

Implementing Lists

- next():

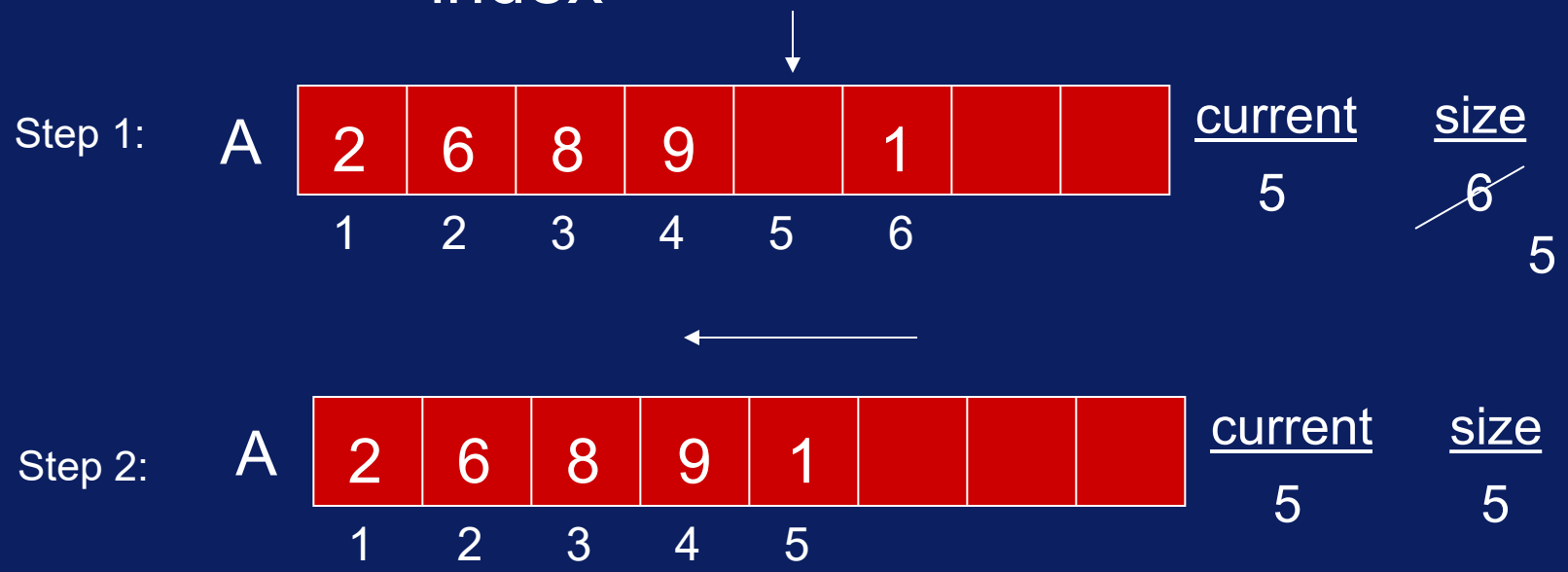
Implementing Lists

- remove(): removes the element at the current index



Implementing Lists

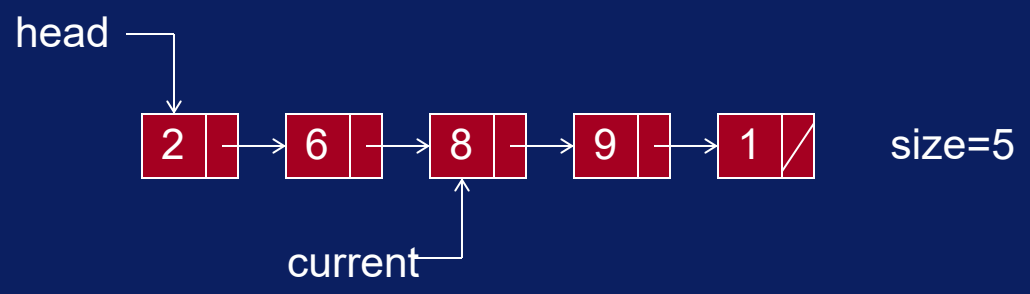
- remove(): removes the element at the current index



- We fill the blank spot left by the removal of 7 by shifting the values to the right of position 5 over to the left one space.

Linked List

- Picture of our list (2, 6, 8, 9, 1) stored as a linked list:



Linked List

Note some features of the list:

- Need a *head* to point to the first node of the list. Otherwise we won’t know where the start of the list is.

Linked List

Note some features of the list:

- Need a *head* to point to the first node of the list. Otherwise we won't know where the start of the list is.
- The *current* here is a pointer, not an index.

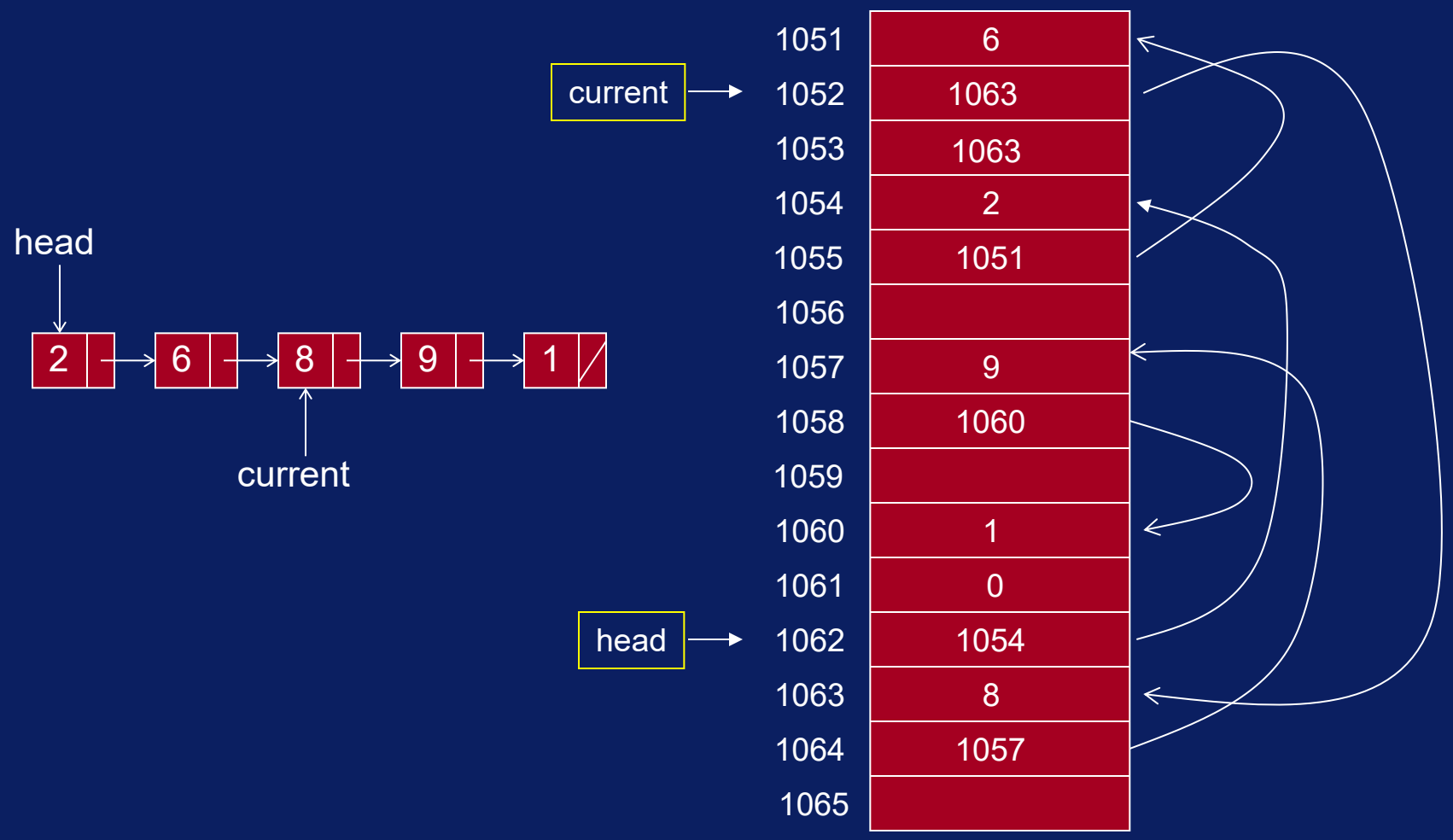
Linked List

Note some features of the list:

- Need a *head* to point to the first node of the list. Otherwise we won't know where the start of the list is.
- The *current* here is a pointer, not an index.
- The next field in the last node points to *nothing*. We will place the memory address NULL which is guaranteed to be inaccessible.

Linked List

- Actual picture in memory:

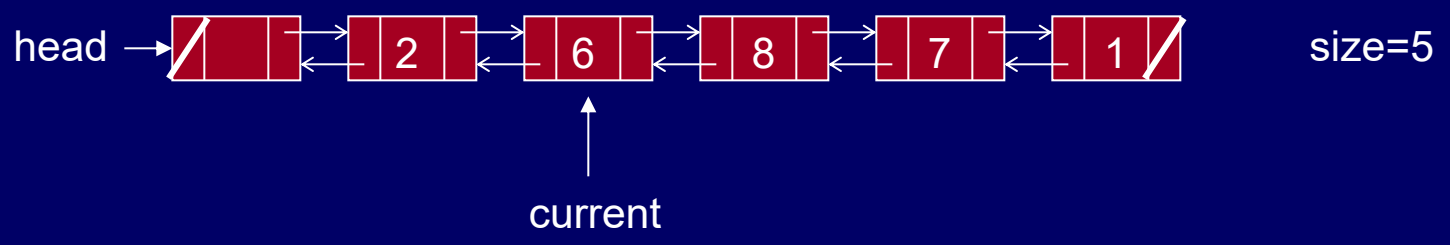


Doubly-linked List OR Two Way Link List

- Moving forward in a singly-linked list is easy; moving backwards is not so easy.
- To move back one node, we have to start at the head of the singly-linked list and move forward until the node before the current.
- To avoid this we can use *two* pointers in a node: one to point to next node and another to point to the previous node:

Doubly-linked List

- Need to be more careful when adding or removing a node.
- Consider add: the order in which pointers are reorganized is important:
- The list can be traversed either forward or backward.

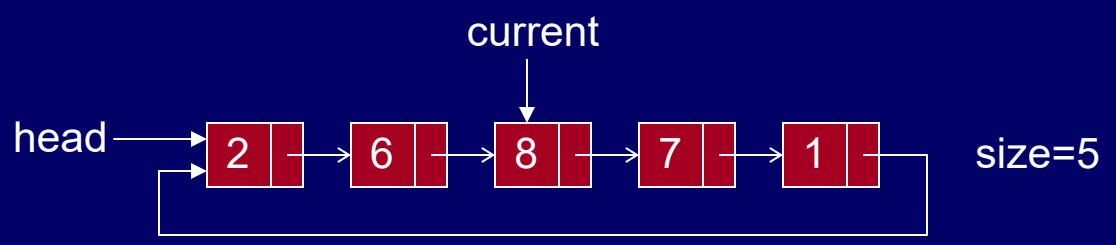
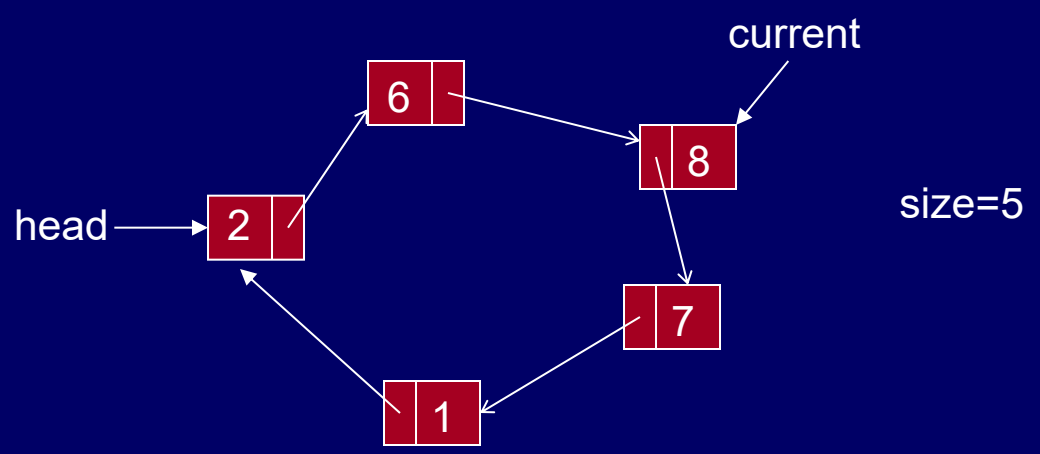


Points to Remember

- Doubly Linked List are more convenient than Singly Linked
- We can traverse in both directions and display the contents in the whole List
- In Doubly Linked List we can traverse from Head to Tail as well as Tail to Head
- Each Node contains two fields, called Links , that are references to the previous and to the Next Node in the sequence of Nodes.
- The previous link of the first node and the next link of the last node points to NULL.

Circular-linked lists

- The next field in the last node in a singly-linked list is set to NULL.
- Moving along a singly-linked list has to be done in a watchful manner.
- Doubly-linked lists have two NULL pointers: prev in the first node and next in the last node
- A way around this potential hazard is to link the last node with the first node in the list to create a *circularly-linked list*.

Cicularly Linked List

- Two views of a circularly linked list: